INFORME TÉCNICO: SESIÓN S08

Sesión: S08 - Mensajería asíncrona entre servicios

Nombre del Estudiante: Ruben Mamani

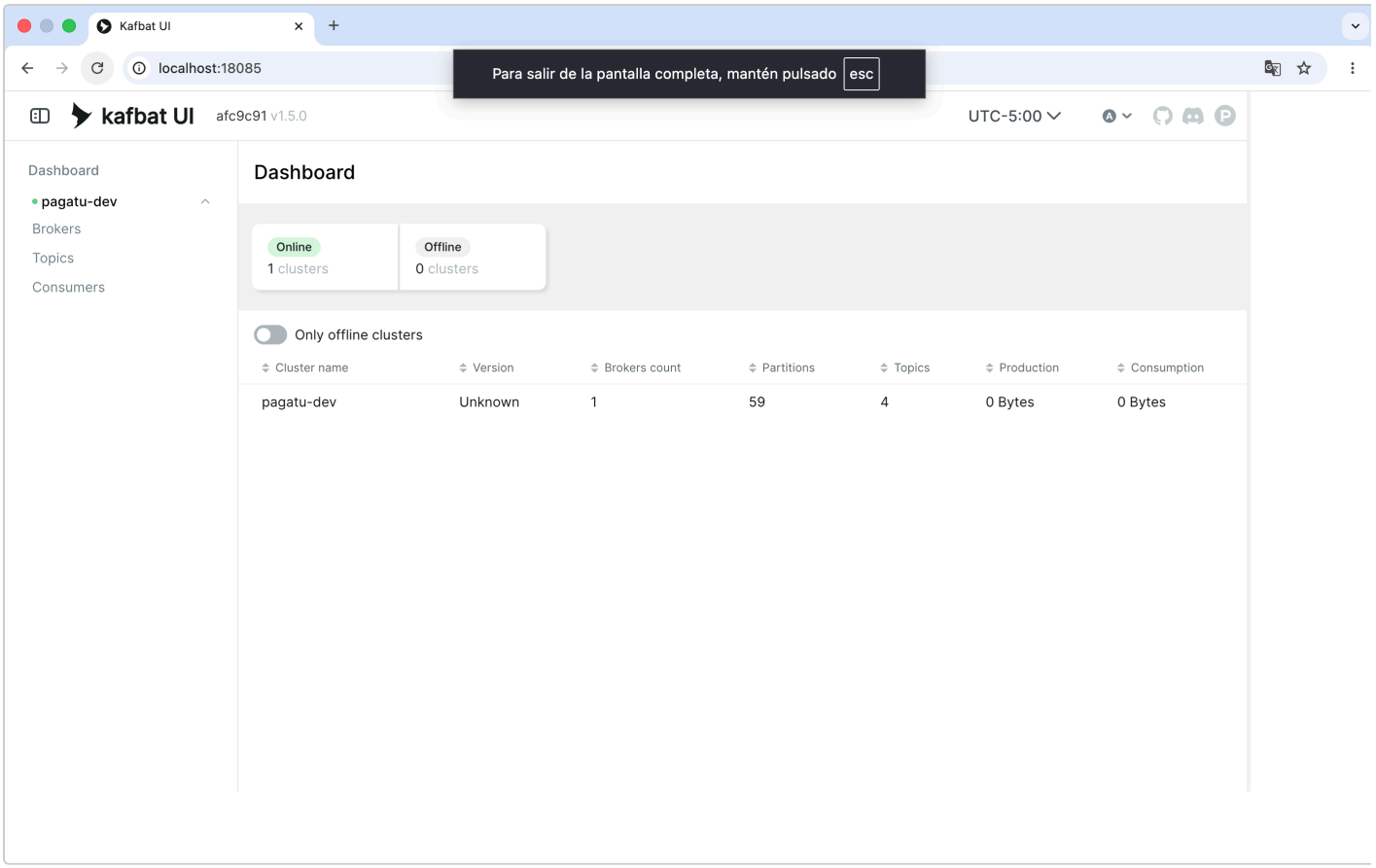
Equipo: Equipo #6

Rol o Aporte Realizado: Arquitectura e implementación de mensajería asíncrona basada en eventos con Apache Kafka, integración de `pagatu-pago-ms`, flujo event-driven (`orden.creada` y `pago.validado`), evento adicional propio (`comprobante.enviado`) y prueba de desacople y lag acumulado en Kafka UI.

Repositorio GitHub: https://github.com/rbn-69-cod/pagatu.git

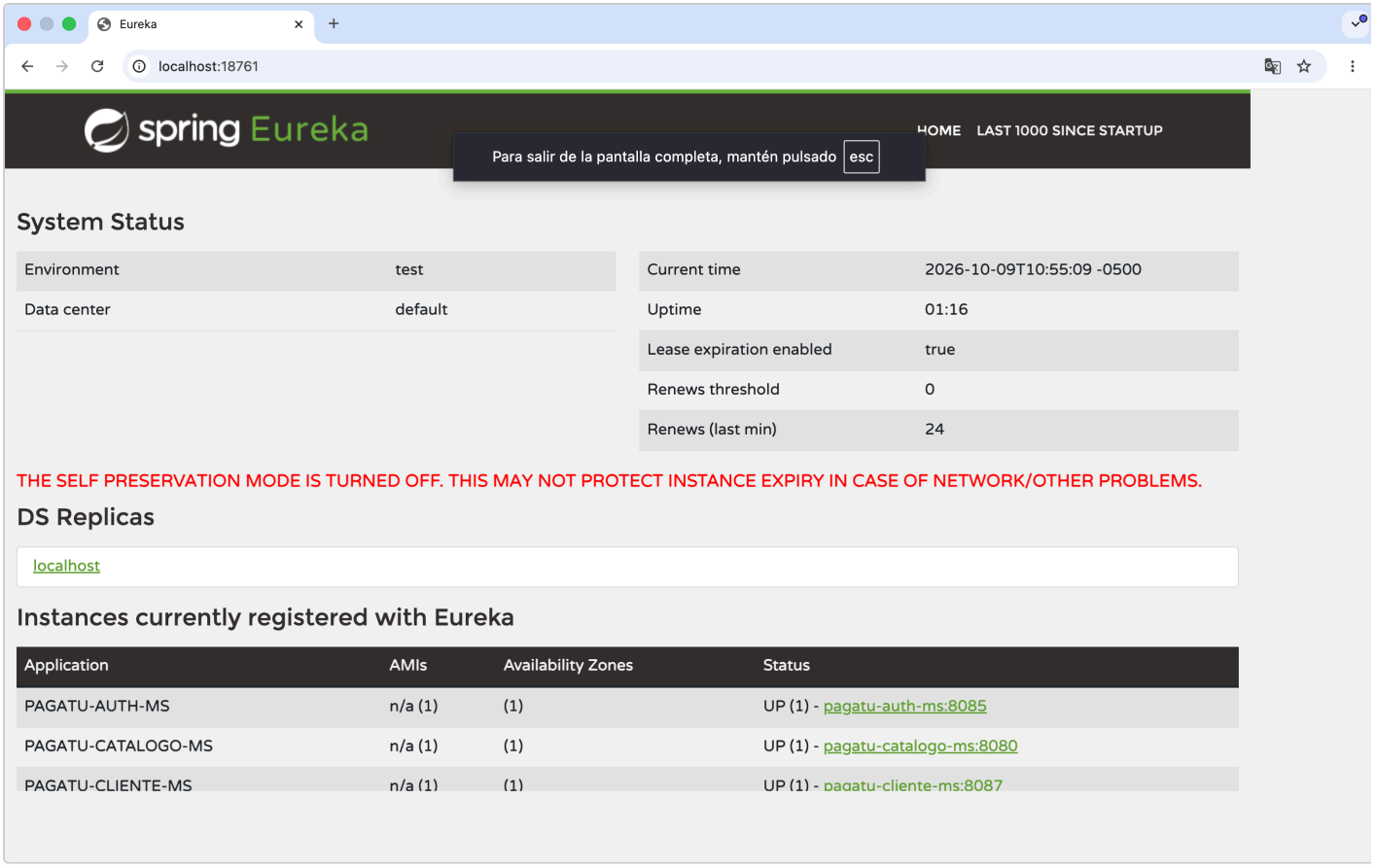
1. BLOQUE 1: KAFKA Y PAGATU-PAGO-MS CONSTRUIDOS

> Captura real de pantalla de mi Macbook mostrando los contenedores de Docker (Apache Kafka en puerto 19092, Kafka UI en puerto 18085), los topics creados (`orden-eventos`, `pago-eventos`, `comprobante-eventos`), el microservicio `pagatu-pago-ms` registrado exitosamente en Eureka (puerto 8086) y la tabla `pagos` en PostgreSQL (puerto 15435).



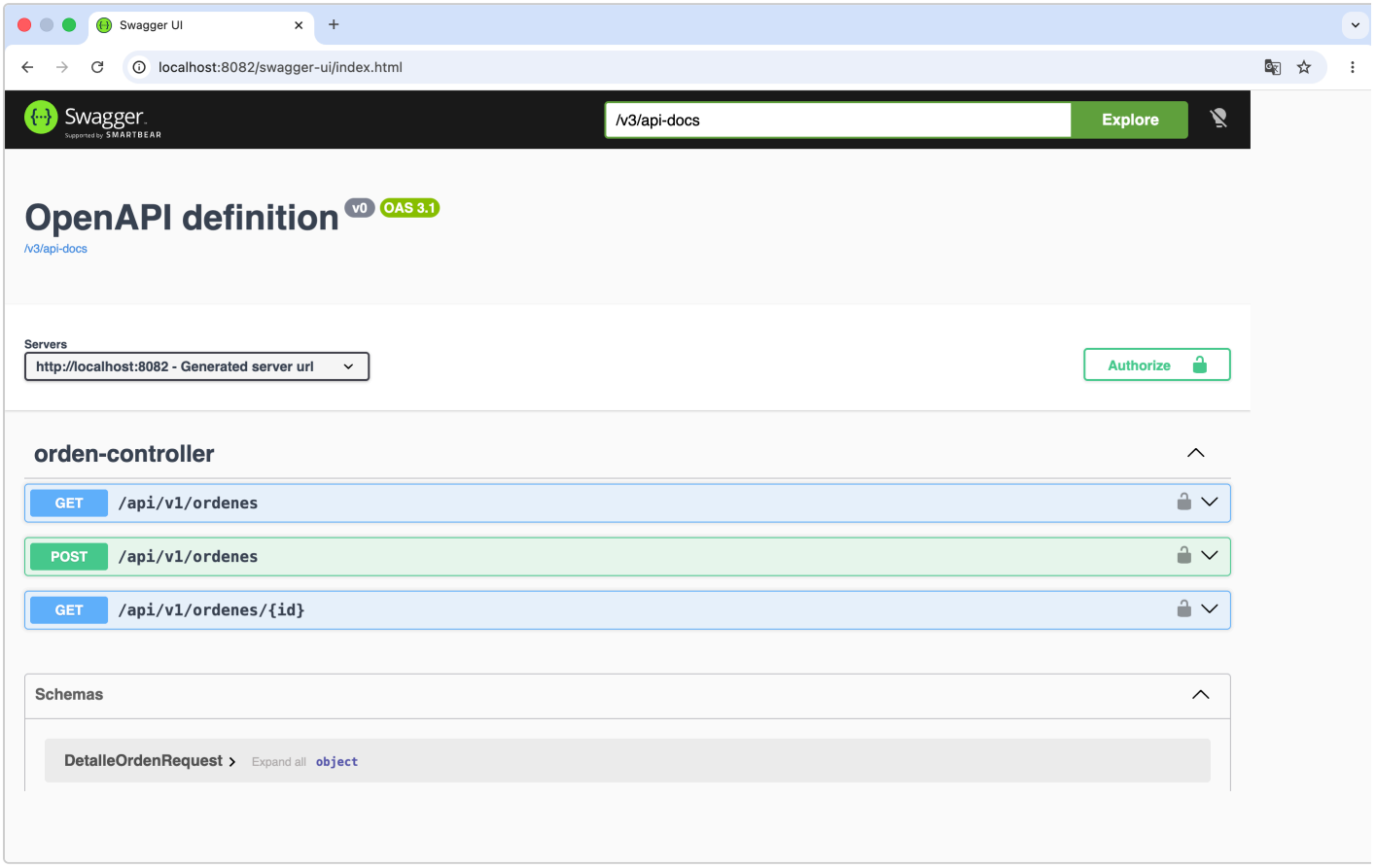
2. BLOQUE 2: EVENTOS ENTRE PAGATU-ORDEN-MS Y PAGATU-PAGO-MS

> Captura real en mi Macbook de la publicación del evento `orden.creada` desde `pagatu-orden-ms` tras registrar una nueva orden vía Gateway. `pagatu-pago-ms` consume asíncronamente el evento, procesa el pago registrándolo en base de datos y publica `pago.validado`. Finalmente, `pagatu-orden-ms` consume dicho mensaje y actualiza el estado de la orden de `PENDIENTE_PAGO` a `PAGADA`.


Eureka
localhost:18761
spring Eureka
HOME
LAST 1000 SINCE STARTUP
Para salir de la pantalla completa, mantén pulsado esc
System Status
Environment
test
Data center
default
Current time
2026-10-09T10:55:09 -0500
Uptime
01:16
Lease expiration enabled
true
Renews threshold
0
Renews (last min)
24
THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.
DS Replicas
localhost
Instances currently registered with Eureka
Application
AMIs
Availability Zones
Status
PAGATU-AUTH-MS
n/a (1)
(1)
UP (1) - pagatu-auth-ms:8085
PAGATU-CATALOGO-MS
n/a (1)
(1)
UP (1) - pagatu-catalogo-ms:8080
PAGATU-CLIENTE-MS
n/a (1)
(1)
UP (1) - pagatu-cliente-ms:8087

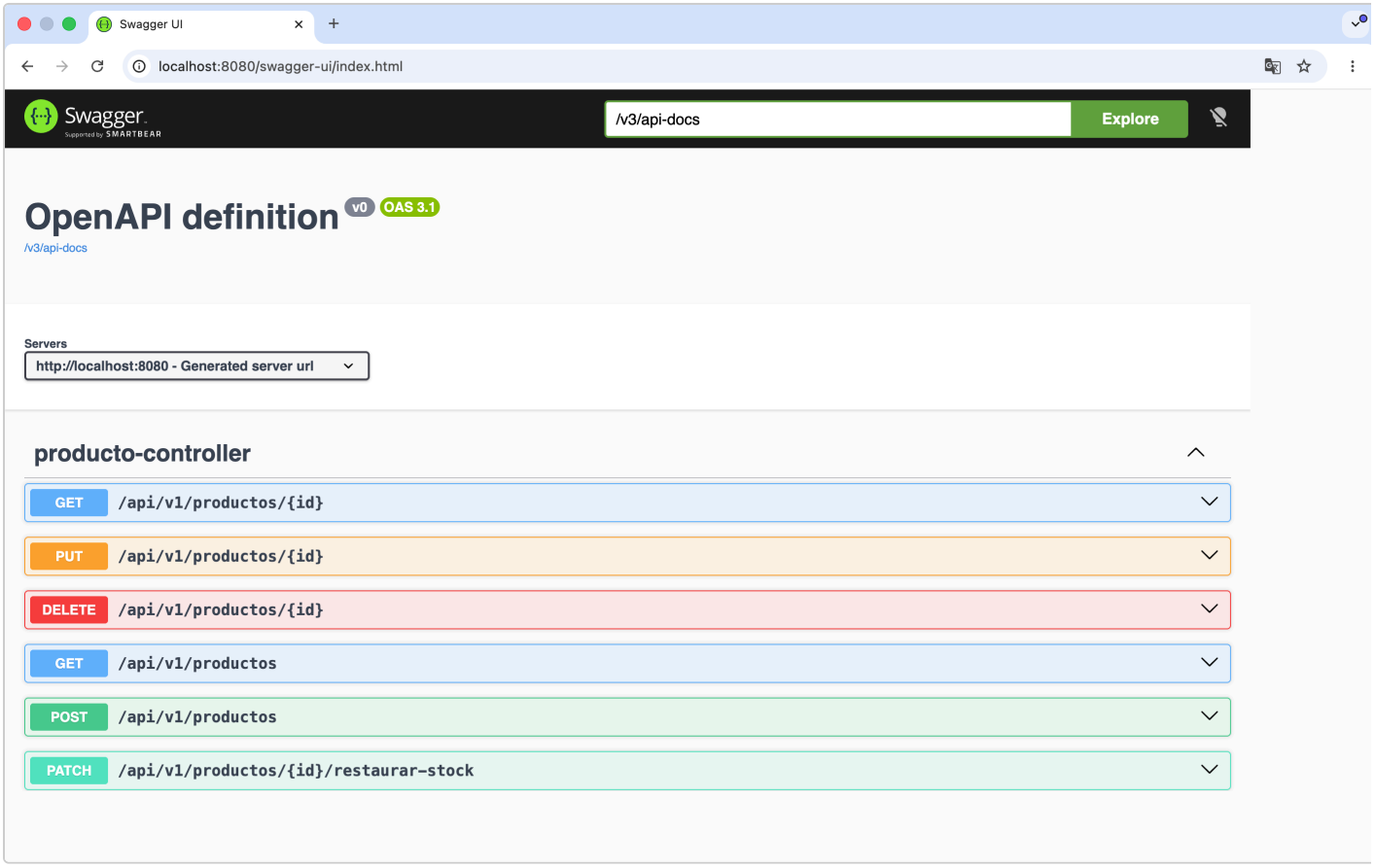
3. BLOQUE 3: DESACOPLE EVIDENCIADO

> Prueba de desacople temporal: Se detuvo el servicio consumidor `pagatu-pago-ms` y se registró una nueva orden. El evento `orden.creada` se acumuló en el broker Kafka manteniendo un **Lag de 1** visible en Kafka UI sin bloquear la respuesta de la orden. Al encender nuevamente `pagatu-pago-ms`, el consumidor procesó el evento pendiente de la cola automáticamente, transicionando la orden al estado `PAGADA`.



4. BLOQUE 4: EVENTO DE NEGOCIO ADICIONAL PROPIO

> **Evento Adicional Integrado:** `comprobante.enviado`
>
> **Contrato del Evento:**
>
> - **Nombre del evento:** `comprobante.enviado`
> - **Topic:** `comprobante-eventos`
> - **Key:** `ordenId` (String)
> - **Estructura y Tipos:** `ordenId` (Long, ID de la orden), `idCliente` (Long, ID del cliente), `montoTotal` (BigDecimal, importe cobrado), `comprobanteUrl` (String, enlace PDF/XML), `fechaEmision` (LocalDateTime, timestamp), `canal` (String, "WHATSAPP").
>
> **Flujo y Desacople:** Al confirmarse el pago en `pagatu-pago-ms`, se publica `comprobante.enviado`. El microservicio `pagatu-cliente-ms` escucha el topic con su propio `group-id` (`pagatu-cliente-ms-group`) para notificar al cliente vía WhatsApp. Se probó el desacople apagar `pagatu-cliente-ms`, verificando la acumulación de mensajes y su consumo al reconectar.



5. ERROR O HALLAZGO REAL

> **Descripción del Error:** Al arrancar los consumidores de Kafka en los microservicios, la consola arrojaba repetidamente `org.springframework.kafka.listener.ListenerExecutionFailedException: Caused by: org.apache.kafka.common.errors.SerializationException`.
>
> **Causa Raíz:** La propiedad de deserialización de Spring Kafka `spring.json.trusted.packages` en los archivos `application.yml` no tenía configurados los paquetes de clases de eventos entre microservicios, bloqueando la conversión del payload JSON a las clases Java correspondientes.
>
> **Solución Aplicada:** Se configuró `spring.json.trusted.packages: "pe.edu.upeu.*"` en los repositorios de configuración centralizada en `pagatu-config`, permitiendo que los consumidores deserialicen correctamente todos los paquetes de eventos del dominio.

6. REFLEXIÓN TÉCNICA BREVE

> El evento `orden.creada` tuvo que incluir los datos esenciales del negocio (`ordenId` , `idCliente` , `montoTotal` , `metodoPago` y los `detalles` de productos). Si el evento solo hubiera transportado el número de orden (`ordenId`), `pagatu-pago-ms` se habría visto obligado a realizar una llamada HTTP síncrona de regreso a `pagatu-orden-ms` para consultar el monto a cobrar. Esto habría destruido el desacople temporal y espacial entre los microservicios, reintroduciendo una dependencia fuerte y aumentando el riesgo de cascada de fallos si `pagatu-orden-ms` estuviera indisponible o experimentara alta latencia en ese momento.

7. ANEXO: FEEDBACK DE LA SESIÓN

¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?	El concepto de desacople asíncrono y resiliencia en arquitecturas event-driven: los servicios publican hechos pasados a un broker (Kafka) y continúan sin esperar a los consumidores.
¿Qué punto de la clase te resultó más confuso o te dejó con dudas?	El manejo de garantías de entrega ("at least once") y cómo implementar idempotencia en el consumidor cuando Kafka reentrega un mensaje duplicado.
¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?	¿Cuál es el patrón recomendado para gestionar transacciones distribuidas y compensaciones cuando un pago falla (Patrón Saga)?
Nivel de comprensión de la clase de hoy:	**[X] ¡Entendido! - Lo domino y podría explicarlo.** [ ] Más o menos. [ ] Necesito ayuda.
¿Cómo puedo ayudarte a comprender mejor el tema?	Incluyendo un diagrama visual de las particiones, offsets y consumer groups en Apache Kafka.
Autoevaluación de participación y esfuerzo:	**[X] Muy Comprometido/a: Me esforcé al máximo.** [ ] Comprometido/a. [ ] Poco Comprometido/a.
Mi satisfacción con la clase fue (1 al 10):	**10 / 10**